

COURSE CODE/TITLE:	CPE212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	09/03/2026	DATE SUBMITTED:	09/03/2026
NAME:	Christopher T. Fegalan	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 TARGETING SPECIFIC NODES

1. PROCEDURE OUTPUT

TASK 1:

Server 3 (CentOS)	192.168.56.117 / 192.168.0.103 (Home)
Server 1 (Ubuntu)	192.168.56.120 / 192.168.0.102 (Home)
Server 2 (Ubuntu)	192.168.56.119 / 192.168.0.104 (Home)

1. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.

fegalanvm@fegalan-workstation: ~/CPE212_fegalan

GNU nano 6.2

site.yml

```

---
- hosts: all
  become: true
  tasks:
    - name: install apache and php for Ubuntu Servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS Servers
      dnf:
        name:
          - httpd
          - php

```

Read 22 lines

^G Help

^X Exit

^O Write Out

^R Read File

^W Where Is

^L Replace

^K Cut

^U Paste

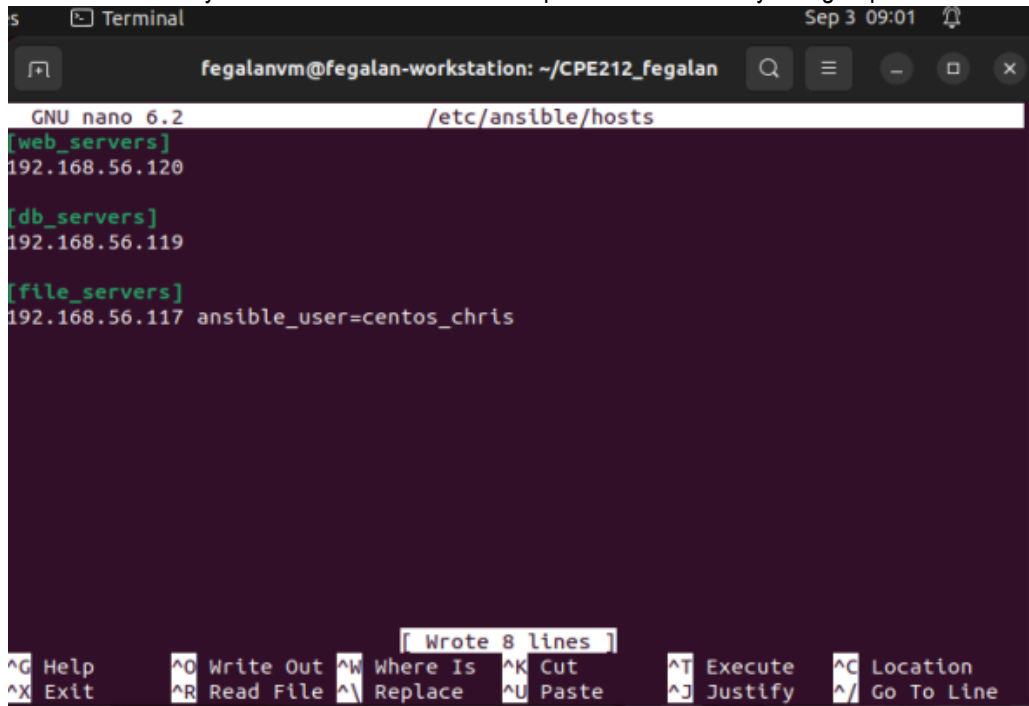
^T Execute

^J Justify

^C Location

^_ Go To Line

2. Edit the inventory file. Remove the variables we put in our last activity and group:



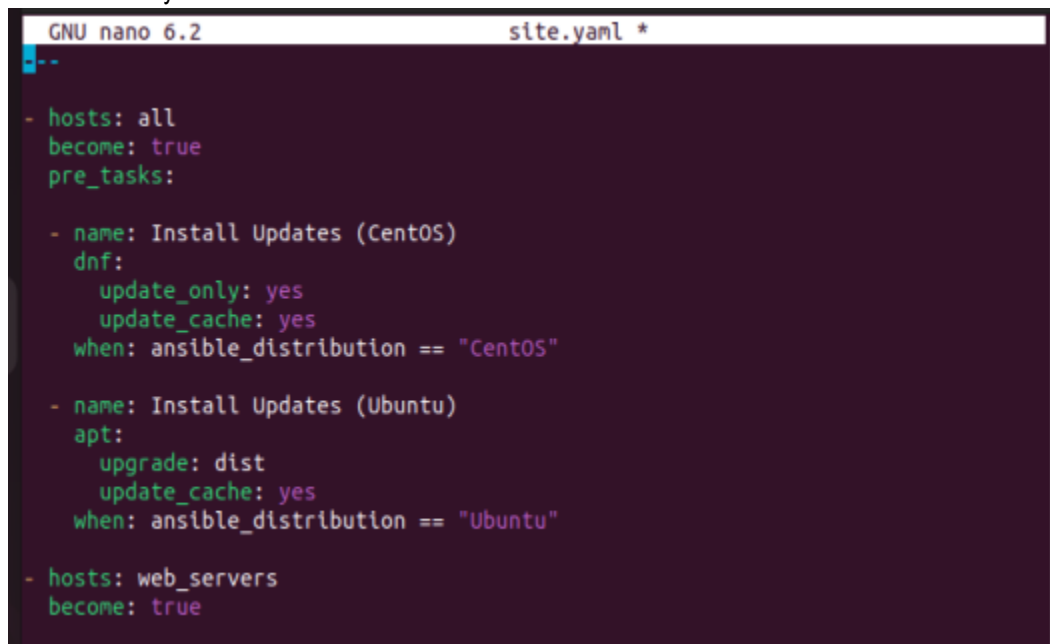
```
GNU nano 6.2 /etc/ansible/hosts
[web_servers]
192.168.56.120

[db_servers]
192.168.56.119

[file_servers]
192.168.56.117 ansible_user=centos_chris

Wrote 8 lines
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

3. Edit the site.yaml:



```
GNU nano 6.2 site.yaml *
--
- hosts: all
  become: true
  pre_tasks:

    - name: Install Updates (CentOS)
      dnf:
        update_only: yes
        update_cache: yes
      when: ansible_distribution == "CentOS"

    - name: Install Updates (Ubuntu)
      apt:
        upgrade: dist
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true
```

```
fegalanvm@fegalan-workstation:~/CPE212_fegalan$ ansible-playbook site.yml --as
-become-pass
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
[DEPRECATION WARNING]: Distribution centos 9 on host 192.168.56.117 should use
/usr/libexec/platform-python, but is using /usr/bin/python for backward
compatibility with prior Ansible releases. A future Ansible release will
default to using the discovered platform python for this host. See https://docs
.ansible.com/ansible/2.10/reference_appendices/interpreter_discovery.html for
more information. This feature will be removed in version 2.12. Deprecation
warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
ok: [192.168.56.117]
ok: [192.168.56.118]
ok: [192.168.56.119]

TASK [Install Updates (CentOS)] *****
skipping: [192.168.56.118]
skipping: [192.168.56.119]
```

```
PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.120]

TASK [install apache and php for Ubuntu Servers] *****
ok: [192.168.56.120]

TASK [install apache and php for CentOS Servers] *****
skipping: [192.168.56.120]

PLAY RECAP *****
192.168.56.117 : ok=2 changed=0 unreachable=0 failed=0 s
skipped=1 rescued=0 ignored=0
192.168.56.119 : ok=2 changed=0 unreachable=0 failed=0 s
skipped=1 rescued=0 ignored=0
192.168.56.120 : ok=4 changed=0 unreachable=0 failed=0 s
skipped=2 rescued=0 ignored=0
```

Run the site.yml file and describe the result:

- It did the pre_tasks first before moving on to the actual tasks. It skipped the two ips for the first part of the pre_tasks since I defined it to only run on CentOS distributions and for the actual tasks since i only put the webserver metagroup it only used the remote machine belonging to that group.

4. Let's try to edit again the site.yml file. This time, we are going to add plays targeting the other servers. This time we target the db_servers by adding it on the current site.yml. Below is an example: (Note add this at the end of the playbooks from task 1.3.

```
GNU nano 0.2 site.yml
tasks:

- name: Install mariadb package (CentOS)
  yum:
    name: mariadb-server
    state: latest
    when: ansible_distribution == "CentOS"

- name: "Mariadb - Restarting/Enabling"
  service:
    name: mariadb
    state: restarted
    enabled: true

- name: Install mariadb package (Ubuntu)
  apt:
    name: mariadb-server
    state: latest
    when: ansible_distribution == "Ubuntu"

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^I Paste     ^J Justify   ^_ Go To Line
```

```
fegalanvm@fegalan-workstation: ~/CPE212_fegalan
ok: [192.168.56.120]

TASK [install apache and php for CentOS Servers] *****
skipping: [192.168.56.120]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.119]

TASK [Install mariadb package (CentOS)] *****
skipping: [192.168.56.119]

TASK [Mariadb - Restarting/Enabling] *****
fatal: [192.168.56.119]: FAILED! => [{"changed": false, "msg": "Could not find the requested service mariadb: host"}]

PLAY RECAP *****
192.168.56.117      : ok=1    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.56.119      : ok=2    changed=0    unreachable=0    failed=1    s
kipped=1    rescued=0    ignored=0
192.168.56.120      : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
```

Run the site.yml file and describe the result.

- It failed to run because 119 or my ubuntu/db_server does not have mariadb.

6. Edit the site.yml again. This time we will append the code to configure installation on the file_servers group. We can add the following on our file.

```
fegalanvm@fegalan-server2: ~
fegalanvm@fegalan-server2:~$ systemctl status mariadb
Unit mariadb.service could not be found.
fegalanvm@fegalan-server2:~$
```

Describe the output:

- Like i mentioned previously it didnt install and skipped the 119 ip since it is a ubuntu distro so when running this command it failed and mariadb wasn't installed. It didn't run the next task of installing it on Ubuntu since ansible takes out failed machines automatically after just one failed attempt during its sequence of runtime.

```
TASK [Install mariadb package (CentOS)] *****
skipping: [192.168.56.119]

TASK [Mariadb - Restarting/Enabling] *****
fatal: [192.168.56.119]: FAILED! => {"changed": false, "msg": "Could not find the requested service mariadb: host"}

PLAY RECAP *****
192.168.56.117      : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.168.56.119      : ok=2    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.120      : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Describe the output:

- It didn't run the file_servers play since the previous play had a failed task subsequently stopping all the plays after to fail. Since the db_servers only had 1 remote server that failing means all the tasks and plays after it also fails. Ansible stops a not working ip/failed ip if it failed at that task but should still proceed to the next task/play if it had other ips that are working but since 100% of the ip in db_servers failed everything as well stopped.

TASK 2:

```

GNU nano 6.2 site.yaml *
tasks:
- name: Install mariadb package (CentOS)
  tags: centos,db,mariadb
  dnf:
    name: mariadb-server
    state: latest
    when: ansible_distribution == "CentOS"
- name: "Mariadb - Restarting/Enabling CentOS"
  service:
    name: mariadb
    state: restarted
    enabled: true
    when: ansible_distribution == "CentOS"
- name: Install mariadb package (Ubuntu)
  tags: db,mariadb,ubuntu
  apt:
    name: mariadb-server
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^_ Go To Line

```

Make sure to save the file and exit.

```

TASK [Mariadb - Restarting/Enabling CentOS] *****
skipping: [192.168.0.104]

TASK [Install mariadb package (Ubuntu)] *****
changed: [192.168.0.104]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.103]

TASK [Install Samba Package] *****
ok: [192.168.0.103]

PLAY RECAP *****
192.168.0.102      : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.0.103      : ok=4    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.0.104      : ok=4    changed=1    unreachable=0    failed=0    s
kipped=3    rescued=0    ignored=0
user123@workstation:~/CPE212_fegalan$

```

Run the site.yaml file and describe the result: It was able to run successfully. I added a `ansible_distribution` to the `ubuntu` in `db_servers` just so that it would pass the play.

On the local machine, try to issue the following commands and describe each result:

2.1 ansible-playbook --list-tags site.yml

```
user123@workstation:~/CPE212_fegalan$ ansible-playbook --list-tags site.yml

playbook: site.yml

  play #1 (all): all    TAGS: []
    TASK TAGS: [always]

  play #2 (web_servers): web_servers    TAGS: []
    TASK TAGS: [apache, apache2, centos, httpd, ubuntu]

  play #3 (db_servers): db_servers    TAGS: []
    TASK TAGS: [centos, db, mariadb, ubuntu]

  play #4 (file_servers): file_servers    TAGS: []
    TASK TAGS: [samba]
```

- Shows every tag in every play.

2.2 ansible-playbook --tags centos --ask-become-pass site.yml

```
ok: [192.168.0.102]
ok: [192.168.0.104]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.102]

TASK [install apache and php for CentOS Servers] *****
skipping: [192.168.0.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.104]

TASK [Install mariadb package (CentOS)] *****
skipping: [192.168.0.104]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.103]
```

- I noticed that it only run the plays with the tasks of centos excluding the pre_tasks that has a tag of always which always runs no matter the tag specification unless specified not to.

2.3 ansible-playbook --tags db --ask-become-pass site.yml

```

user123@workstation: ~/CPE212_fegalan
TASK [Install mariadb package (CentOS)] *****
skipping: [192.168.0.104]

TASK [Install mariadb package (Ubuntu)] *****
ok: [192.168.0.104]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.103]

PLAY RECAP *****
192.168.0.102      : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.0.103      : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.0.104      : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0

user123@workstation:~/CPE212_fegalan$ nano site.yml
user123@workstation:~/CPE212_fegalan$

```

- It only runs the tasks with the tag db

2.4 ansible-playbook --tags apache --ask-become-pass site.yml

```

TASK [Install Updates (Ubuntu)] *****
skipping: [192.168.0.103]
ok: [192.168.0.102]
ok: [192.168.0.104]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.102]

TASK [install apache and php for Ubuntu Servers] *****
ok: [192.168.0.102]

TASK [install apache and php for CentOS Servers] *****
skipping: [192.168.0.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.104]

PLAY [file_servers] *****

```

- It only runs the tasks with the tag apache

2.5 ansible-playbook --tags "apache,db" --ask-become-pass site.yml

```
PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.102]

TASK [install apache and php for Ubuntu Servers] *****
ok: [192.168.0.102]

TASK [install apache and php for CentOS Servers] *****
skipping: [192.168.0.102]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.0.104]

TASK [Install mariadb package (CentOS)] *****
skipping: [192.168.0.104]

TASK [Install mariadb package (Ubuntu)] *****
ok: [192.168.0.104]

PLAY [file_servers] *****
```

- It only runs the tasks with the tag db, and apache

TASK 3:

Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```
user123@workstation: ~/CPE212_fegalan
GNU nano 7.2 site.yml

- name: start httpd (CentOS)
  tags: apache, centos, httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"

hosts: db_servers
become: true
tasks:

- name: Install mariadb package (CentOS)
  tags: centos,db,mariadb
  dnf:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"

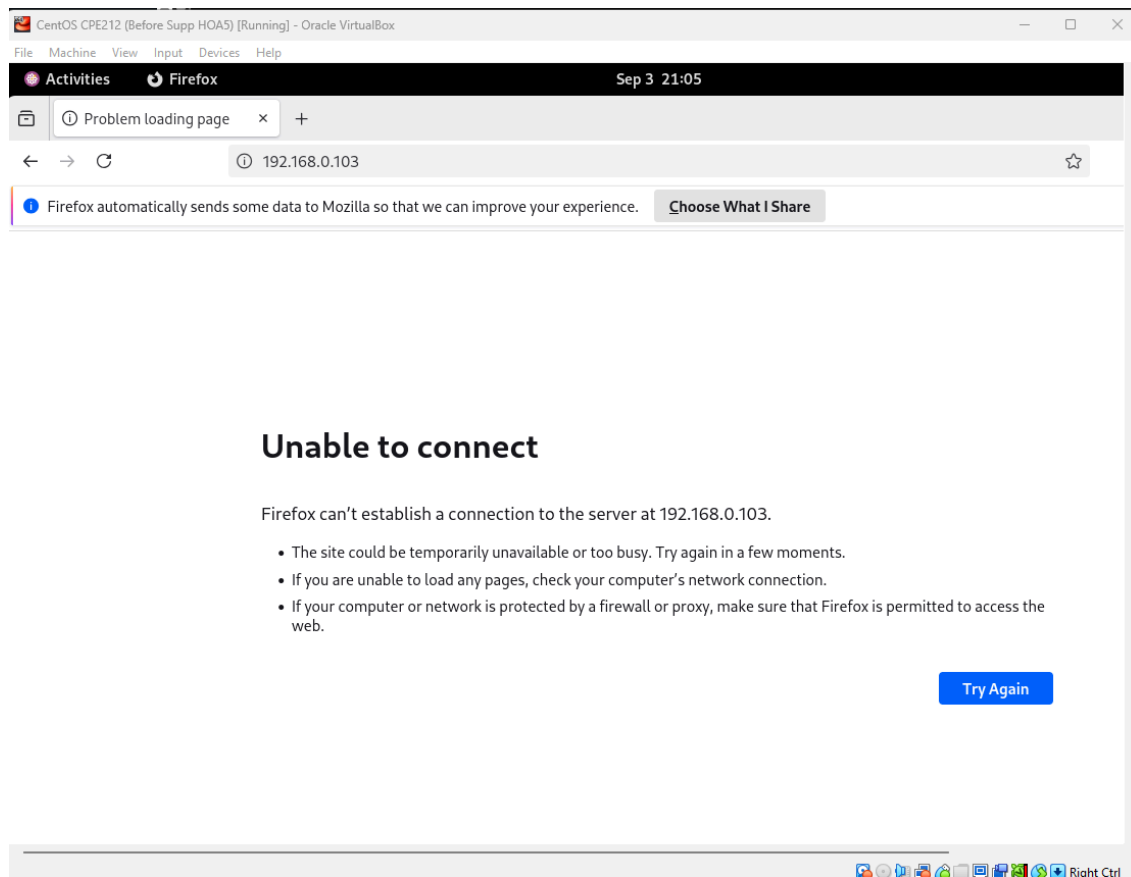
- name: "Mariadb - Restarting/Enabling CentOS"

[ Wrote 83 lines ]

^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace  ^U Paste    ^J Justify  ^_ Go To Line
```

To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command `sudo systemctl stop httpd`. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.

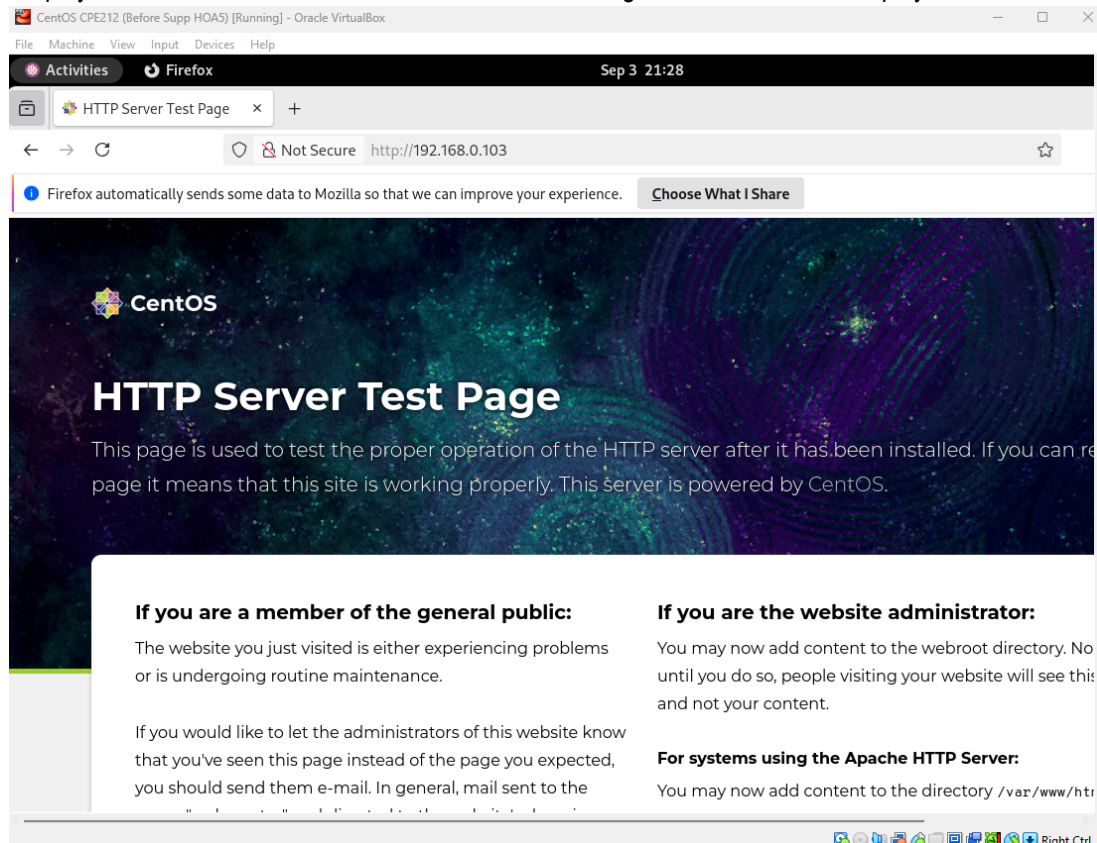
```
[centos_chris@localhost ~]$ sudo systemctl stop httpd
[centos_chris@localhost ~]$ sudo systemctl status httpd
○ httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: d>
   Active: inactive (dead)
   Docs: man:httpd.service(8)
```



```
TASK [install apache and php for CentOS Servers] *****
changed: [192.168.0.103]

TASK [start httpd (CentOS)] *****
changed: [192.168.0.103]
```

Go to the local machine and this time, run the site.yml file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result. To automatically enable the service every time we run the playbook, use the command enabled: true similar to Figure 7.1.2 and save the playbook.



- It ran the httpd service which enabled the httpd of CentOS.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:

1. Creating a Role for Web Servers
2. Creating a Role for Web Servers
3. Creating a Role for Database Servers
4. Combining Multiple Roles in a Playbook

To orchestrate multiple roles in a single playbook for different server groups. Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for Database.

I created roles.

```
user123@workstation:~/CPE212_fegalan$ nano suppack6.yaml
user123@workstation:~/CPE212_fegalan$ mkdir -p roles/webserver/tasks roles/db/tasks
user123@workstation:~/CPE212_fegalan$ nano roles/webserver/tasks/main.yaml
user123@workstation:~/CPE212_fegalan$ nano roles/db/tasks/main.yaml
```

Yaml file for role webserver.

```

user123@workstation: ~/CPE212_fegalan/roles/webserver/tasks
GNU nano 7.2 main.yaml
---
- name: Install apache2 (Ubuntu)
  tags: apache, web, ubuntu
  apt:
    name: apache2
    state: present
    update_cache: yes
  when: ansible_distribution == "Ubuntu"

- name: Start apache2 (Ubuntu)
  tags: apache, web, ubuntu
  service:
    name: apache2
    state: started
    enabled: true
  when: ansible_distribution == "Ubuntu"

```

Yaml file for role db.

```

user123@workstation: ~/CPE212_fegalan
GNU nano 7.2 roles/db/tasks/main.yaml
---
- name: Install mariadb-server (Ubuntu)
  tags: db, mariadb, ubuntu
  apt:
    name: mariadb-server
    state: present
    update_cache: yes
  when: ansible_distribution == "Ubuntu"

- name: Start mariadb (Ubuntu)
  tags: db, mariadb, ubuntu
  service:
    name: mariadb
    state: started
    enabled: true
  when: ansible_distribution == "Ubuntu"

```

Defined the playbook

```
GNU nano 7.2                                supp6.yaml
e- --
- name: Configure web server
  hosts: web_servers
  become: true
  roles:
    - webserver

- name: Configure database server
  hosts: db_servers
  become: true
  roles:
    - db
```

Tried test cases such as all tags and using the different tags defined in the playbook supp6.

```
PLAY [Configure web server] *****

TASK [Gathering Facts] *****
ok: [192.168.0.102]

TASK [webserver : Install apache2 (Ubuntu)] *****
ok: [192.168.0.102]

TASK [webserver : Start apache2 (Ubuntu)] *****
ok: [192.168.0.102]

PLAY [Configure database server] *****

TASK [Gathering Facts] *****
ok: [192.168.0.104]

TASK [db : Install mariadb-server (Ubuntu)] *****
ok: [192.168.0.104]

TASK [db : Start mariadb (Ubuntu)] *****
ok: [192.168.0.104]

PLAY RECAP *****
192.168.0.102      : ok=3    changed=0    unreachable=0    failed=0    ski
pped=0    rescued=0    ignored=0
192.168.0.104      : ok=3    changed=0    unreachable=0    failed=0    ski
```

ansible-playbook supp6.yaml

- It ran every play

```

user123@workstation:~/CPE212_fegalan$ ansible-playbook supp6.yaml --tags web --ask
-become-pass
BECOME password:

PLAY [Configure web server] *****

TASK [Gathering Facts] *****
ok: [192.168.0.102]

TASK [webserver : Install apache2 (Ubuntu)] *****
ok: [192.168.0.102]

TASK [webserver : Start apache2 (Ubuntu)] *****
ok: [192.168.0.102]

PLAY [Configure database server] *****

TASK [Gathering Facts] *****
ok: [192.168.0.104]

PLAY RECAP *****
192.168.0.102      : ok=3    changed=0    unreachable=0    failed=0    skip=0
192.168.0.104      : ok=1    changed=0    unreachable=0    failed=0    skip=0

```

ansible-playbook supp6.yaml --tags web

- It ran the tasks with the tag web. It still ran the database server play but didn't perform any task since there were no web tags.

```

user123@workstation:~/CPE212_fegalan$ ansible-playbook supp6.yaml --tags db --ask
-become-pass
BECOME password:

PLAY [Configure web server] *****

TASK [Gathering Facts] *****
ok: [192.168.0.102]

PLAY [Configure database server] *****

TASK [Gathering Facts] *****
ok: [192.168.0.104]

TASK [db : Install mariadb-server (Ubuntu)] *****
ok: [192.168.0.104]

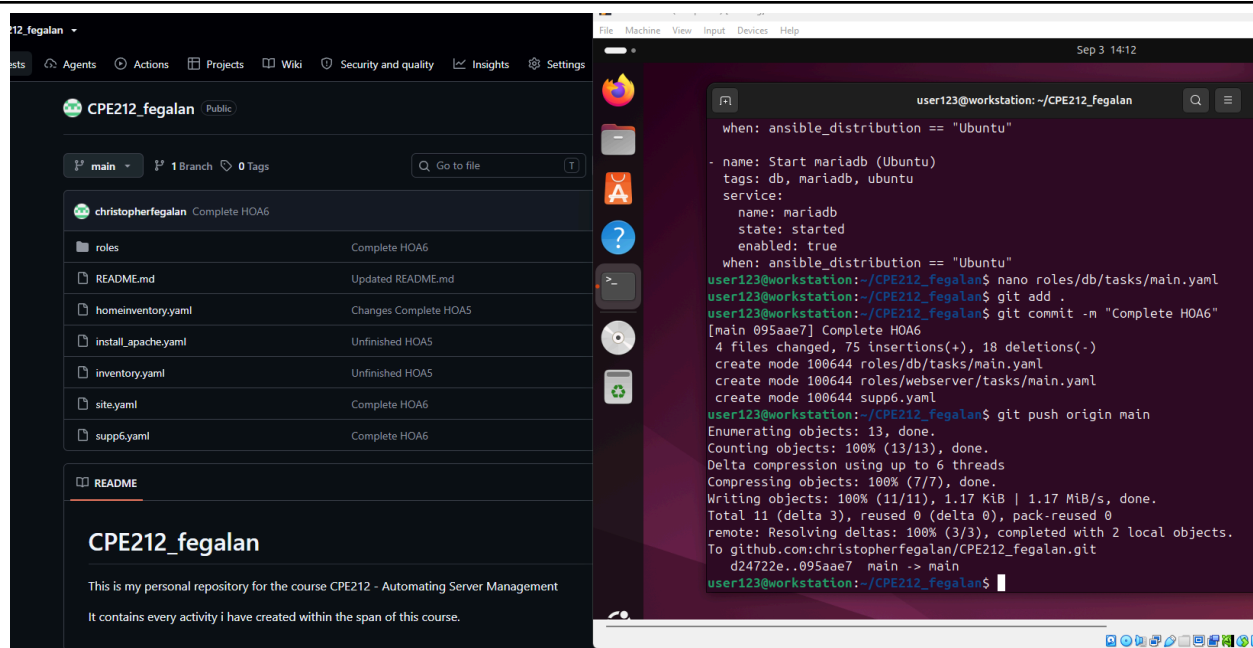
TASK [db : Start mariadb (Ubuntu)] *****
ok: [192.168.0.104]

PLAY RECAP *****
192.168.0.102      : ok=1    changed=0    unreachable=0    failed=0    skip=0
192.168.0.104      : ok=3    changed=0    unreachable=0    failed=0    skip=0

```

ansible-playbook supp6.yaml --tags db

- Same as the previous explanation but instead with the tag db.



- Saved to github

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?

- It simplifies running the servers. Instead of running them one by one individually or typing a list of ip. You can use metagroups to just call the group and it will remember every ip in that group and use it.

2. What is the importance of tags in playbooks?

- It simplifies running tasks, and it is an easier way of running specified tasks such as installation or centos specific tags that only runs the tasks needed or specified by the tag while going through each play.

3. Why do you think some services need to be managed automatically in playbooks?

- Managing services automatically ensures that the applications are always up and running when they should be. It guarantees that if a server reboots or crashes, the system will restart the service on its own without needing a person to log in and turn it back on. It also ensures the service stays running consistently every time you run your automation tools.

Conclusions/Learnings:

- This activity was really helpful in getting to know ansible better. Using tags to only run specific commands reduces runtime of commands and makes tasks very specific to that kind of automation. Using groups to group ip and assign them to a group so that when you need to call a certain group of ip it would be simplified instead of calling them one by one you just need to call their metagroup and it will then list out every ip that belongs to that metagroup. Services helped me understand better how to manage background processes such as starting and restarting httpd within this activity to allow and disable access when needed. Overall this activity really expanded what i know about ansible and all the mistakes and errors that i have made during this activity further reinforced what I know about ansible and playbooks in general.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT



I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Debugging

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: It was used to explain and expand on problems I personally had with Ubuntu and trying to set up each step between the tasks. Telling me why it happened and how I can solve it.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.